

Deep Learning

NST-CS-DL501 | Semester 5 • 2026 | Complete Visual Cheatsheet & Revision Compendium

TOTAL LECTURES 12 Comprehensive Units	TOTAL PAGES 13 Pages (Cover + Notes)	COURSE TRACK Semester 5 Examination Prep
--	---	---

TABLE OF CONTENTS & INDEX

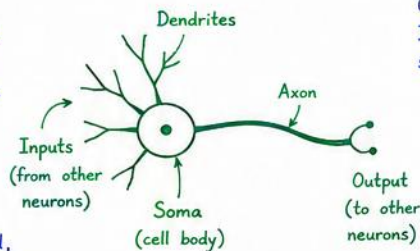
UNIT	LECTURE TOPIC & SUMMARY	PAGE
L-01	Where it Begins: The Neuron & Decision Boundaries	Page 02
L-02	The Activation Function and the Update Rule	Page 03
L-03	Modern Activations & Vanishing Gradients	Page 04
L-04	Multilayer Perceptrons & Backpropagation	Page 05
L-05	Optimization & Learning-Rate Control	Page 06
L-06	Gradient Descent Mechanics & Loss Landscapes	Page 07
L-07	Optimizers with Memory: Momentum, NAG & Adam	Page 08
L-08	Adaptive Learning Rates: AdaGrad, RMSProp & Adam	Page 09
L-09	Regularisation: L2, L1, Early Stopping & Dropout	Page 10
L-10a	CNN Foundations: From Pixels to Patterns	Page 11
L-10b	Inside LeNet, AlexNet & VGG-16	Page 12
L-11	Modern Architectures: Inception & ResNet	Page 13

1 The Biological & Mathematical Neuron

1.1 Biology of the Neuron

In 1888, Santiago Ramón y Cajal showed brain is made of individual neurons that process info in a unidirectional manner.

- Receive:** collects chemical & electrical signals via dendrites.
- Sum:** aggregates incoming potentials at cell body (soma).
- Fire:** if cumulative potential > threshold, generates action potential down axon to signal other neurons.



1.2 Mathematical Formulation

Convert biological mechanism into math. Incoming signals are modeled as weighted sum (strength of connection).

Mathematical Neuron Model

$$\text{Output} = 1 \quad \text{if} \quad \sum_{i=1}^n w_i x_i \geq \theta$$

$$\text{Output} = 0 \quad \text{if} \quad \sum_{i=1}^n w_i x_i < \theta$$

Where:

x_i : Input signals

w_i : Weights (importance of inputs)

θ (Theta): Activation threshold

2 McCulloch & Pitts Model (1943)

2.1 Logic Gates as Building Blocks

By stacking simple sum-and-threshold nodes, they built logic gates → basic components of any computer capable of arithmetic, memory & computation.

2.2 Gate Implementations

Gate	Weights (w)	Threshold ($\theta \geq$)	Truth Table ($x_1, x_2 \rightarrow y$)															
AND	$w_1 = 1$ $w_2 = 1$	2	<table><tr><th>x_1</th><th>x_2</th><th>y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x_1	x_2	y	0	0	0	0	1	0	1	0	0	1	1	1
x_1	x_2	y																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR	$w_1 = 1$ $w_2 = 1$	1	<table><tr><th>x_1</th><th>x_2</th><th>y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x_1	x_2	y	0	0	0	0	1	1	1	0	1	1	1	1
x_1	x_2	y																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT	$w = -1$ (single input)	0	<table><tr><th>x</th><th>y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	y	0	1	1	0									
x	y																	
0	1																	
1	0																	

2.3 Implications of the 1943 Result

Since one cell can act as AND, OR, or NOT gate, a network of these units can construct any logical circuit whatsoever.

Real Talk: The Missing Element

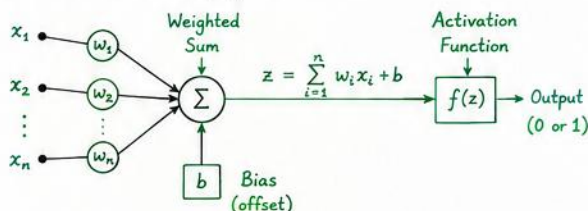
Weights had to be set manually by humans. Model could compute, but did NOT learn from experience. True learning $\Rightarrow$ weights must adapt from data automatically.

3 Perceptron Forward & Backward Propagation

3.1 Forward Propagation

Two phases:

1. Linear Combination: compute total score.
2. Activation Function: compare with threshold to output 0 or 1.



3.2 The Concept of Bias

Original: $w_1 x_1 + w_2 x_2 \geq \theta$

Rearrange: $w_1 x_1 + w_2 x_2 - \theta \geq 0$

Let $b = -\theta$ (bias)

$$w_1 x_1 + w_2 x_2 + b \geq 0$$

Bias acts as an offset $\rightarrow$ simplifies computation.

3.3 Backward Propagation & Learning

If prediction $\hat{y} \neq$ true label y , update weights & bias to reduce error.

Perceptron Learning Rule

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i$$

$$b \leftarrow b + \eta (y - \hat{y})$$

y = true label (0 or 1)
 $\hat{y}$ = predicted output (0 or 1)
 η = learning rate (small +ve value)
 Update moves parameters in direction that decreases the error.

Activation Functions

- Step Function: $f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$
- Sigmoid: $f(z) = \frac{1}{1 + e^{-z}}$
- In Perceptron: Step func. used to give hard output (0 or 1).

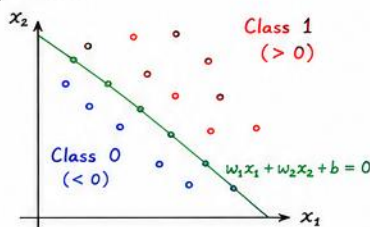
4 Decision Boundary & Linear Separation

4.1 Geometry of the Boundary

Perceptron draws a straight line (decision boundary) in 2D feature space.

- Above line: $w_1 x_1 + w_2 x_2 + b > 0 \rightarrow$ Class 1
- Below line: $w_1 x_1 + w_2 x_2 + b < 0 \rightarrow$ Class 0
- On line: $w_1 x_1 + w_2 x_2 + b = 0 \rightarrow$ Decision Boundary

Since inputs are to first power (linear), boundary is a straight line $\rightarrow$ linear separation.



4.2 A Concrete Mathematical Example

Choose: $w_1 = 2, w_2 = 3, b = -12$

Decision rule: $2x_1 + 3x_2 - 12 \geq 0$

Boundary line: $2x_1 + 3x_2 - 12 = 0$

Intercepts:

- If $x_2 = 0 \Rightarrow x_1 = 6$ (x-intercept)
- If $x_1 = 0 \Rightarrow x_2 = 4$ (y-intercept)

Points above line $\rightarrow$ positive sum (Class 1)
 Points below line $\rightarrow$ negative sum (Class 0)

5 The XOR Limitation

5.1 The Non-Linear Separation Problem

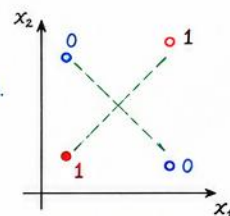
Single perceptron uses linear combination $\rightarrow$ can draw only a straight line. If classes need diagonal grouping, no single straight line can separate them.

XOR Truth Table

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

5.2 Minsky & Papert Proof (1969)

- Minsky & Papert proved that a single perceptron cannot learn XOR (Exclusive OR).
- XOR requires diagonal separation which is impossible with any straight line.
- This result $\rightarrow$ funding dried up for more than a decade (end of first wave of neural network enthusiasm).



Critical Bottleneck

Straight-line boundary is baked into linear combination ($w \cdot x + b$). Changing activation functions (step $\rightarrow$ sigmoid) DOES NOT remove this limitation.

Conclusion: A single perceptron is powerful but limited by linearity. Non-linear problems like XOR require Multi-layer networks.

DEEP LEARNING FOUNDATIONS

LECTURE 02 – THE ACTIVATION FUNCTION AND THE UPDATE RULE

Black – Main | Red – Key/Formula

Blue – Explanation | Green – Example/Diagram

1. WHAT AN ACTIVATION FUNCTION DOES

1.1 ARITHMETIC VS. ACTIVATION

- Perceptron does 2 operations in sequence:
 - Arithmetic: compute raw score $z = wx + b$
 - Activation function: produce prediction $\hat{y} = f(z)$
- Without activation, output is just a measurement, not a classified answer.

1.2 INTUITIVE EXAMPLE: STUDENT PASS/FAIL

Let $w = 1$, $b = -40$ (marks x)

Student	Marks (x)	Raw Score $z = x - 40$
A	65	$65 - 40 = 25$
B	32	$32 - 40 = -8$

Need binary answer (pass=1, fail=0). Activation function converts raw score to final classification.

Threshold (Step) Decision:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{else} \end{cases}$$

Apply:

A: $f(25) = 1 \Rightarrow$ Pass

B: $f(-8) = 0 \Rightarrow$ Fail

2. THE GRADIENT DESCENT UPDATE RULE

2.1 MATHEMATICAL UPDATE FORMULATION

- Bad prediction $\Rightarrow$ large loss. Use squared error loss:

$$L = \frac{1}{2} (\hat{y} - y)^2$$

- To reduce loss, change weights opposite to gradient.

Gradient descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w}$$

α : learning rate
 y : target

2.2 CHAIN RULE DERIVATION

- Since $w \rightarrow z \rightarrow \hat{y} \rightarrow L$, use chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Derive each term:

$$\frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$$

$$\frac{\partial \hat{y}}{\partial z} = f'(z)$$

$$\frac{\partial z}{\partial w} = x$$

Multiply:

Complete update equation:

$$w_{\text{new}} = w_{\text{old}} - \alpha (\hat{y} - y) f'(z) x$$

Step size controlled by 3 factors: error $(\hat{y} - y)$, input (x) , and slope of activation $f'(z)$.

3. WHY THE STEP FUNCTION CANNOT LEARN

3.1 STEP FUNCTION DERIVATIVE

- For step (threshold) function:

$$z > 0 \Rightarrow f(z) = 1 \Rightarrow f'(z) = 0$$

$$z < 0 \Rightarrow f(z) = 0 \Rightarrow f'(z) = 0$$

$$z = 0 \Rightarrow \text{jump} \Rightarrow f'(0) \text{ undefined}$$

Thus, $f'(z) = 0$ for all defined z (ie., $z \neq 0$).

3.2 NUMERICAL SIMULATION (Step Function)

Let $x = 2$, $w = 1$, $b = 0$, $y = 0$, $\alpha = 0.5$

Step	Calculation	Value
① $z = wx + b$	$(1)(2) + 0$	$= 2$
② $\hat{y} = f(z)$	$f(2)$	$= 1$
③ Error $(\hat{y} - y)$	$1 - 0$	$= 1$
④ Derivative $f'(z)$	$f'(2)$	$= 0$
⑤ Gradient $\frac{\partial L}{\partial w}$	$(1)(0)(2)$	$= 0$
⑥ Update w_{new}	$1 - (0.5)(0)$	$= 1$ (no change)

Even with max error (1), gradient = 0 $\Rightarrow$ weight doesn't change!
Increasing α won't help, since $\alpha \cdot 0 = 0$.

4. THE SIGMOID FUNCTION

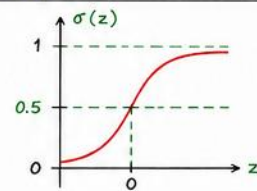
4.1 DEFINITION & GEOMETRY

- Sigmoid (logistic) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Smooth S-curve passing through $(0, 0.5)$.

- Output strictly between 0 and 1.



4.2 DERIVATIVE DERIVATION

- Write $\sigma(z) = (1 + e^{-z})^{-1}$

- By chain rule: $\sigma'(z) = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z})$

- Inner derivative: $\frac{d}{dz}(1 + e^{-z}) = -e^{-z}$

- Substitute: $\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$

- Use $e^{-z} = (1 + e^{-z}) - 1 \Rightarrow \sigma'(z) = \frac{(1 + e^{-z}) - 1}{(1 + e^{-z})^2}$

- Split: $\sigma'(z) = \frac{1}{1 + e^{-z}} - \frac{1}{(1 + e^{-z})^2} = \sigma(z) - \sigma^2(z)$

$$\therefore \sigma'(z) = \sigma(z) (1 - \sigma(z))$$

Max $\sigma'(z)$
at $z = 0$
 $\Rightarrow 0.25$

5. WORKED EXAMPLE: WEIGHT UPDATE WITH SIGMOID

5.1 SIGMOID NUMERICAL EXAMPLE

Use: $x = 2$, $w = 1$, $b = 0$, $y = 0$, $\alpha = 0.5$

Step	Calculation	Value (approx)
① $z = wx + b$	$(1)(2) + 0$	$= 2$
② $\hat{y} = \sigma(z)$	$\sigma(2) = \frac{1}{1 + e^{-2}}$	$= 0.880797$
③ Error $(\hat{y} - y)$	$0.880797 - 0$	$= 0.880797$
④ Derivative $\sigma'(z)$	$\sigma(2)(1 - \sigma(2))$	$= 0.104994$
⑤ Gradient $\frac{\partial L}{\partial w}$	$(0.880797)(0.104994)(2)$	$= 0.184956$
⑥ Update w_{new}	$1 - (0.5)(0.184956)$	$= 0.907522$

5.2 DIRECTION & SIGN VALIDATION

- Target $y = 0$, prediction $\hat{y} = 0.88$ (too high).
- To lower $\hat{y}$, need to lower raw score z (sigmoid $\uparrow$ with z).
- $z = wx + b$, $x = 2$ (positive) $\Rightarrow$ lower w to lower z .
- w decreased: $1 \rightarrow 0.907522 \Rightarrow$ update moves correct way.

6. THE TANH FUNCTION

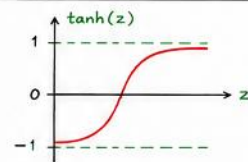
6.1 DEFINITION & OUTPUT RANGE

- Hyperbolic Tangent (tanh):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- S-shaped, passes through origin.

- Output range: $(-1, 1)$.



6.2 DERIVATIVE DERIVATION

- Let $u = e^z - e^{-z}$, $v = e^z + e^{-z} \Rightarrow \tanh(z) = u/v$

- $u' = e^z + e^{-z} = v$, $v' = e^z - e^{-z} = u$

- Quotient rule: $\frac{d}{dz}(\frac{u}{v}) = \frac{u'v - uv'}{v^2}$

$$\therefore \tanh'(z) = \frac{v^2 - u^2}{v^2} = 1 - \frac{u^2}{v^2} = 1 - \tanh^2(z)$$

Max $\tanh'(z)$
at $z = 0$
 $\Rightarrow 1.00$

6.3 NUMERICAL EXAMPLE WITH TANH

Let $x = 1$, $w = 0$, $b = 0$, $y = 1$, $\alpha = 1$

Step	z	$\hat{y} = \tanh(z)$	Error $(\hat{y} - y)$	$\tanh'(z)$	Gradient $(\hat{y} - y) \tanh'(z) x$	w_{new}
0	0	0	$0 - 1 = -1$	$1 - 0^2 = 1$	$(-1)(1)(1) = -1$	$0 - (1)(-1) = 1$

7. COMPARISON AND SUMMARY

7.1 SIGMOID VS. TANH PEAK HEIGHTS (Same params:

$x = 1$, $w = 0$, $b = 0$, $y = 1$, $\alpha = 1$)

Activation	Error $(\hat{y} - y)$	Max $f'(z)$ at $z = 0$	Step Size $ \alpha(\hat{y} - y)f'(z)x $
Sigmoid	-0.5	0.25	0.125
Tanh	-1	1.00	1.000

Tanh step is 8x larger!

- Error ratio: 2 ($|-1|$ vs $|-0.5|$)

- Derivative ratio: 4 (1.00 vs 0.25)

- Product: $2 \times 4 = 8$

(Faster learning)

7.2 SUMMARY TABLE OF ACTIVATION FUNCTIONS

Activation	Formula $f(z)$	Derivative $f'(z)$	Max f'	Learns?
Threshold (Step)	1 if $z \geq 0$ 0 else	0 ($z \neq 0$)	0.00	No $\times$
Sigmoid	$\frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	0.25	Yes $\checkmark$ (slowly)
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$	1.00	Yes $\checkmark$ (faster)

★ **KEY TAKEAWAY:** The derivative term $f'(z)$ acts as a gatekeeper in neural updates. If $f'(z)$ is flat at 0, learning cannot take place.

Tanh enables faster learning than Sigmoid by maintaining larger derivatives and errors around the origin.

Executive Summary & Formula Sheet

- Traditional activations like **Sigmoid** squish values into (0,1), capping derivative $\sigma'(z) \leq 0.25$. When inputs are large ($|z|$ high), $\sigma'(z) \approx 0 \rightarrow$ weight updates freeze (**Vanishing Gradient**).
- Modern architectures use **piece-wise linear** or **exponential** activations (**ReLU**, **Leaky ReLU**, **ELU**) in hidden layers and **Softmax** at the output.

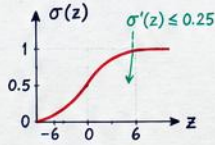
Key Formulas

Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$
ReLU	$f(z) = \max(0, z)$	$f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$
Leaky ReLU	$f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases} \quad (\alpha = 0.01)$	
ELU	$f(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$	
Softmax	$\hat{y}^i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	

1 The Vanishing Gradient Problem

1.1 Sigmoid Derivative Saturation

- Sigmoid**: $\sigma(z) = \frac{1}{1 + e^{-z}}$
 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Maximum derivative at $z = 0$: $\sigma'(0) = 0.25$
- As $|z|$ grows, $\sigma'(z) \rightarrow 0$ exponentially.
- In deep networks, multiplying small gradients (< 0.25) layer after layer $\rightarrow$ gradients vanish $\rightarrow$ early layers stop updating weights.



1.2 Mathematical Proof with Extreme Inputs

- Update rule: $w_{\text{new}} = w_{\text{old}} - \alpha(\hat{y} - y)f'(z)x$
- Let $x = 1, w = 6, b = 0, y = 0, \alpha = 1 \Rightarrow z = 6$
- $\sigma(6) = 0.997521 \Rightarrow$ error $(\hat{y} - y) = 0.997521$
- $\sigma'(6) = 0.997521(1 - 0.997521) = 0.002472$

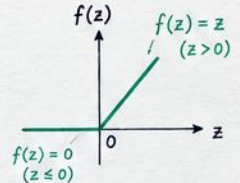
The Contradiction:

- Error was 99.75% of maximum, yet the step was only 0.0025.
- $\Rightarrow$ **Vanishing Gradient**: When $|z|$ is large, sigmoid is flat $\rightarrow \sigma'(z) \approx 0 \rightarrow$ gradient $\approx 0 \rightarrow$ no learning.

2 Rectified Linear Unit (ReLU)

2.1 Definition and Derivative

- ReLU**: $f(z) = \max(0, z)$
 $f'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$
- Derivative is exactly 1 for $z > 0$
 $\rightarrow$ gradients flow back without scaling down eliminates vanishing gradients for active neurons.



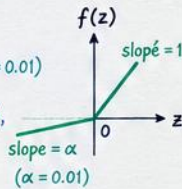
2.2 Dying ReLU Problem

- When $z \leq 0$, derivative is 0.
- If a large negative update pushes weights/bias such that $z < 0$ for all training examples, the neuron always outputs 0 and receives 0 gradient.
- It becomes permanently "dead" and cannot recover.

3 Leaky ReLU & Parametric ReLU

3.1 Fixing the Dead Slope

- Leaky ReLU**: $f(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$ (typically $\alpha = 0.01$)
- Ensures even when $z < 0$, the derivative is $\alpha \neq 0$, preventing neurons from dying completely.



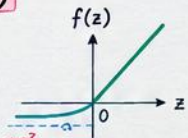
3.2 Parametric ReLU (PReLU)

- Slope α is a learnable parameter updated during backpropagation along with weights.
- Allows the model to determine the optimal leakiness for each layer.

4 Exponential Linear Unit (ELU)

4.1 Definition and Derivative

- ELU**: $f(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$
- Derivative for $z \leq 0$: $f'(z) = f(z) + \alpha = \alpha e^z$
- Provides smooth exponential saturation for negative inputs.



4.2 Mean Shift Reduction

- Brings the average activation closer to zero (unlike ReLU which is strictly non-negative).
- Speeds up learning and reduces bias shift across layers.
- Higher computational cost due to exponential function.

5 Softmax Output Layer

5.1 Probability Distribution Calculation

- For multi-class classification with K classes:
- $$\hat{y}^i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{where} \quad \sum_{i=1}^K \hat{y}^i = 1$$
- Converts raw logits into a normalized probability distribution.

5.2 Worked Example (K=3)

- Given logits: $z_1 = 2.0, z_2 = 1.0, z_3 = 0.1$
- $e^2 = 7.389056, e^1 = 2.718282, e^{0.1} = 1.105171$
- $\sum e^{z_j} = 11.212509$
- $\hat{y}^1 = 7.389056 / 11.212509 = 0.658992$ (65.9%)
- $\hat{y}^2 = 2.718282 / 11.212509 = 0.242433$ (24.2%)
- $\hat{y}^3 = 1.105171 / 11.212509 = 0.098575$ (9.9%)

Output:
valid probability distribution
($0 \leq \hat{y}^i \leq 1$,
 $\sum \hat{y}^i = 1$)

6 Activation Function Comparison & Choice

6.1 Trade-off Matrix

Function	Output Range	Pros	Cons
Sigmoid	(0, 1)	Smooth, probability interpretation	Vanishing gradient, not zero-centered
Tanh	(-1, 1)	Zero-centered	Vanishing gradient at saturation
ReLU	[0, ∞)	Fast, no vanishing gradient ($z > 0$)	Dying ReLU
Leaky ReLU	($-\infty, \infty$)	No dying neurons, robust	Hyperparameter tuning (α)
ELU	($-\infty, \infty$)	No dying neurons, mean shift reduction	Higher computation (exp)

6.2 Practical Selection Guidelines

- Use **ReLU** for hidden layers in standard feedforward and convolutional networks.
- Use **Leaky ReLU** or **ELU** if dead neurons hinder convergence.
- Use **Softmax** for multi-class classification outputs.
- Use **Sigmoid** for binary output layers.

Common Usage

- Hidden layers $\rightarrow$ ReLU / Leaky ReLU / ELU
- Output layer (multi-class) $\rightarrow$ Softmax
- Output layer (binary) $\rightarrow$ Sigmoid

1. Executive Summary & Formula Sheet

Core Concept

- A single perceptron cannot solve non-linearly separable functions like XOR.
- Adding a hidden layer transforms input space into a new feature representation where points become linearly separable.
- Backpropagation uses the chain rule to compute loss gradients w.r.t. every weight across layers.

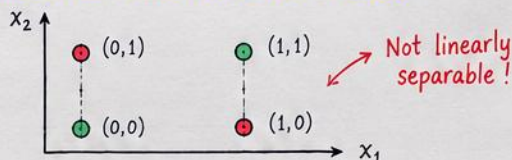
Key Formulas

- Forward pass : $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$
 $a^{[l]} = g(z^{[l]})$
- Parameters : $N_{out} (N_{in} + 1)$
- Output error : $\delta^{[L]} = (\hat{y} - y) \odot g'(z^{[L]})$
- Hidden error : $\delta^{[l]} = (W^{[l+1]T} \delta^{[l+1]}) \odot g'(z^{[l]})$
- Weight gradient : $\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$

2. Resolving XOR with Hidden Layers

2.1 Why single linear boundary fails ?

- XOR outputs 1 for (0,1), (1,0) and 0 for (0,0), (1,1).
- No single straight line $w_1 x_1 + w_2 x_2 + b = 0$ can separate (0,1) and (1,0) from (0,0) and (1,1).
- Convex hulls of the two classes intersect.



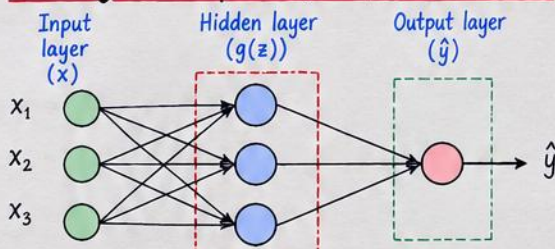
2.2 Combining OR and NAND gates

- XOR can be written as $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$.
- Hidden layer computes OR (h_1) and NAND (h_2), output neuron computes AND(h_1, h_2).

x_1	x_2	OR	NAND	XOR
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

- Hidden layer maps inputs to new space:
 $(0,0) \rightarrow (0,1)$ $(1,1) \rightarrow (1,0)$
 $(0,1) \rightarrow (1,1)$ $(1,0) \rightarrow (1,1) \rightarrow \text{linearly separable!}$

3. Multilayer Perceptron (MLP) Architecture



- Input layer : pass-through nodes (no computation).
- Hidden layer(s) : dense linear combinations + non-linear activations.
- Output layer : produces final prediction $\hat{y}$.
- $W_{ij}^{[l]}$: weight from neuron j in layer $l-1$ to neuron i in layer l .
- $b_i^{[l]}$: bias for neuron i in layer l (shifts linear sum).

4. Parameter Counting Rules

- For a fully connected layer with N_{in} inputs and N_{out} neurons:

$$\text{Total Parameters} = N_{out} (N_{in} + 1) \quad \begin{cases} N_{in} \times N_{out} \text{ weights} \\ N_{out} \text{ biases} \end{cases}$$

- Example** : 3-3-1 network (3 inputs, 3 hidden, 1 output)
 - Hidden layer : (3×3) weights + 3 biases = 12
 - Output layer : (3×1) weights + 1 bias = 4
 - Total parameters = 12 + 4 = **16**

5. Forward Propagation Equations

- For each layer l (starting from $a^{[0]} = x$):

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} ; \quad a^{[l]} = g(z^{[l]})$$

- $z^{[l]}$: linear combination (pre-activation)
- $a^{[l]}$: activation after non-linearity $g(\cdot)$
- Vectorized form for multiple samples :
 $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ → enables fast computation (GPUs)
- Common activations : Sigmoid, ReLU, Tanh etc.

6. Backpropagation Error Derivatives

- 6.1 Output layer gradient (for $L = \frac{1}{2} (\hat{y} - y)^2$)

$$\delta^{[L]} = \frac{\partial L}{\partial z^{[L]}} = (\hat{y} - y) \odot g'(z^{[L]})$$

- 6.2 Hidden layer error signal

$$\delta^{[l]} = (W^{[l+1]T} \delta^{[l+1]}) \odot g'(z^{[l]})$$

- 6.3 Weight gradient

$$\frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T \quad \text{Update: } W^{[l]} \leftarrow W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}}$$

7. Numerical Backpropagation Step-by-Step (Single input, 1 hidden, 1 output)

7.1 Setup

- Input $x = 1$, target $y = 1$
- Learning rate $\alpha = 0.5$
- Weights : $w^{[1]} = 0.5$, $b^{[1]} = 0$
 $w^{[2]} = 0.8$, $b^{[2]} = 0$
- Activation : Sigmoid
 $\sigma(z) = \frac{1}{1 + e^{-z}}$

7.2 Forward pass

- $z^{[1]} = 0.5 \times 1 = 0.5 \Rightarrow a^{[1]} = \sigma(0.5) = 0.6225$
- $z^{[2]} = 0.8 \times 0.6225 = 0.498 \Rightarrow \hat{y} = \sigma(0.498) = 0.6220$
- Loss $L = \frac{1}{2} (0.6220 - 1)^2 = 0.0714$

7.3 Backpropagation

- $\delta^{[2]} = (0.6220 - 1) \cdot \sigma'(0.498) = -0.3780 \cdot (0.6220 \times 0.3780) = -0.0889$
- $\frac{\partial L}{\partial w^{[2]}} = \delta^{[2]} a^{[1]} = -0.0889 \times 0.6225 = -0.0553$
- $w_{new}^{[2]} = 0.8 - 0.5 (-0.0553) = 0.8277$

$$\textcircled{4} \quad \delta^{[1]} = (\delta^{[2]} w^{[2]}) \cdot \sigma'(0.5) \\ = (-0.0889 \times 0.8) \cdot 0.2350 \\ = -0.0167$$

$$\textcircled{5} \quad \frac{\partial L}{\partial w^{[1]}} = \delta^{[1]} x = -0.0167 \times 1 \\ = -0.0167$$

$$\textcircled{6} \quad w_{new}^{[1]} = 0.5 - 0.5 (-0.0167) \\ = 0.5084$$

Gradient Descent : Executive Summary & Formula Sheet

Core Concept :

- Gradient descent updates parameters by moving opposite to the gradient : $w_{t+1} = w_t - \alpha \nabla L(w_t)$
- Choice of batch size balances gradient precision vs computation speed.
- Selecting and scheduling learning rate α is critical : too high $\rightarrow$ divergence, too low $\rightarrow$ stalls convergence.

1 Intuition of Downhill Steps

1.1 Foggy Hill Analogy & Local Slope

- Imagine walking down a foggy mountain blindfolded. You feel the slope under your feet.
- Take small steps in the steepest downhill direction eventually leads to the valley floor (loss minimum).

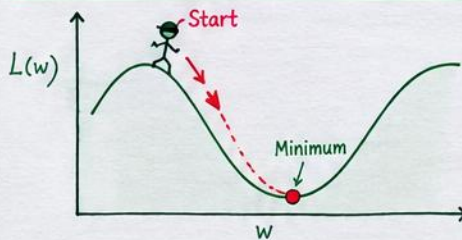
1.2 Mathematical Loss Function Minimization

- The gradient $\nabla L(w)$ points in the direction of steepest loss increase. So, we move in the opposite direction.

$$w_{t+1} = w_t - \alpha \nabla L(w_t)$$

Formula Sheet & Key Rules

- Gradient Descent Update : $w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$
- Batch GD : Uses all N samples (exact, slow).
- Stochastic GD (SGD) : Uses 1 sample (noisy, fast).
- Mini-batch GD : Batch size $B \in [32, 256]$ (balanced GPU parallelization).
- Step Decay : $\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/s \rfloor}$
- Cosine Annealing : $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t / T_{\max}))$



2 Batch GD vs. SGD vs. Mini-batch GD

Full-Batch Gradient Descent

- Uses all N dataset samples per step.
- Exact (true) gradient.
- Smooth convergence for convex loss.
- Requires massive memory and computation per update.

$$\nabla L(w) = \frac{1}{N} \sum_{i=1}^N \nabla L_i(w)$$

Stochastic Gradient Descent (SGD)

- Uses 1 random sample per step.
- High variance $\rightarrow$ noisy trajectory.
- Helps escape shallow local minima.
- Does not converge exactly without learning rate decay.

$$w_{t+1} = w_t - \alpha \nabla L_i(w_t)$$

Mini-batch Gradient Descent

- Uses mini-batch of size B (e.g., 32 or 64).
- Combines GPU vectorization with regularizing noise of SGD.
- Most commonly used in deep learning.

$$w_{t+1} = w_t - \alpha \frac{1}{B} \sum_{i=1}^B \nabla L_i(w_t)$$

Comparison Summary

Method	Batch size	Gradient	Pros	Cons
Batch GD	N	Exact	Smooth	Slow
SGD	1	Noisy	Fast, escape local minima	High variance
Mini-batch GD	32-256	Less noisy	Balanced	Needs tuning

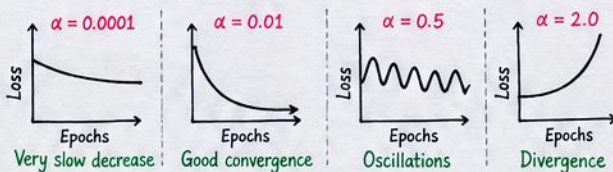
3 Learning Rate Pathology & Diagnosis

3.1 Under-stepping, Oscillation & Divergence

- Too Small** ($\alpha = 0.0001$) : Loss decreases extremely slowly; model gets stuck in plateaus.
- Optimal** ($\alpha = 0.01$) : Smooth logarithmic decline to minimum.
- Too High** ($\alpha = 0.5$) : Loss oscillates across valley walls.
- Very High** ($\alpha = 2.0$) : Steps overshoot, loss explodes to ∞ or NaN.

3.2 Loss Trajectory Visual Signatures

Monitor training loss curves over epochs to diagnose α .

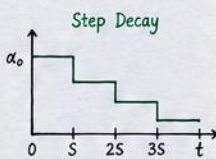


4 Learning-Rate Schedules

4.1 Step Decay & Exponential Decay

- Start with larger α for rapid initial progress.
- Periodically reduce α (e.g., halve every 10 epochs) to settle into precise minima.

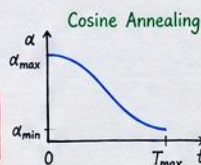
$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/s \rfloor} \quad (\gamma < 1, \text{ e.g., } 0.5)$$



4.2 Cosine Annealing Schedule

- Smoothly decreases α following a cosine curve.

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right) \right)$$



5 Learning Rate Warm-up

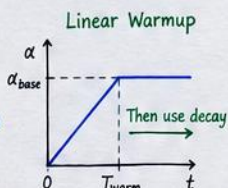
5.1 Why Early High Rates Destabilize Training

- At initialization, random weights produce large, unstable gradients.
- High learning rates immediately can permanently distort early feature representations.

5.2 Linear Warmup Formulation

- Linearly ramp α from 0 to α_{base} over first few epochs (e.g., 5 epochs), allowing gradients to stabilize.

$$\alpha_t = \alpha_{\text{base}} \cdot \frac{t}{T_{\text{warm}}} \quad (0 \leq t \leq T_{\text{warm}})$$



6 Worked Gradient Descent Calculation

6.1 Single Variable Loss $L(w) = (w-3)^2$

- Gradient : $\frac{dL}{dw} = 2(w-3)$
- Target minimum : $w^* = 3$

6.2 Multi-step Convergence Tracing

Starting from $w_0 = 0$, $\alpha = 0.1$:

- Step 1 : $\nabla L(0) = 2(0-3) = -6 \Rightarrow w_1 = 0 - 0.1(-6) = 0.6$
- Step 2 : $\nabla L(0.6) = 2(0.6-3) = -4.8 \Rightarrow w_2 = 1.08$
- Step 3 : $\nabla L(1.08) = 2(1.08-3) = -3.84 \Rightarrow w_3 = 1.464$
- Notice w_t smoothly approaches 3 at rate $(1-2\alpha)^t = 0.8^t$.

1. Core Idea & Intuition

- **Goal:** minimise loss function $f(w)$ by iteratively updating parameters using local information (value + slope) only.
- In high dimensions, global search is infeasible → rely on local exploration.
- Gradient descent moves in **direction of steepest decrease** of loss.

Update rule (Gradient Descent):

$$w_{k+1} = w_k - \alpha \nabla f(w_k)$$

new params learning rate (step size) gradient (slope vector)

Key Formulas

- Gradient vector:

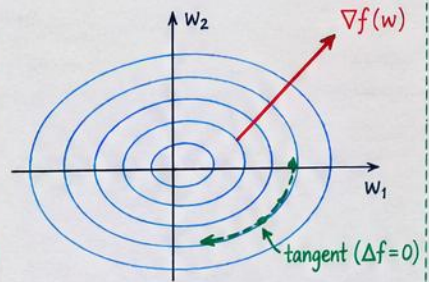
$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$
- Update rule:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$
- Condition number:

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$

(ratio of max to min curvature)
- $\nabla f(x)$ is **perpendicular** (orthogonal) to level contour $f(x) = c$

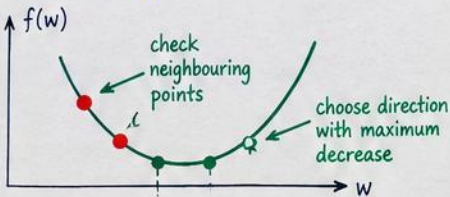
Gradient and Contours



- Contour lines connect points of equal loss ($f(w) = c$).
- Gradient vector is **perpendicular** to the contour line at every point.
- Moving along contour → no change in loss. Moving along $-\nabla f$ → maximum decrease.

2. Local Exploration with Neighbouring Points

- Evaluate loss at nearby points to estimate slope.
- Determine which direction gives maximum decrease per unit distance (directional derivative).
- Forms basis for gradient descent.



3. Direction vs Magnitude of Slope

- **Direction alone is not sufficient.**
 → fixed step in steepest direction ignores how steep the hill is.
- Slope magnitude ($\|\nabla f\|$) acts as a "velocity factor".
 → large steps on steep slopes, small steps near minima ($\|\nabla f\| \rightarrow 0$).
- Thus, **step** = $\alpha \times \nabla f$ (both direction and magnitude matter).

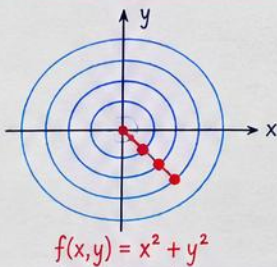
4. Learning Rate (Step Multiplier)

- α (learning rate) scales the gradient to convert slope into spatial step:

$$\Delta w = -\alpha \nabla f$$
- If α too large → overshooting, oscillations, divergence.
- If α too small → very slow convergence.
- In narrow ravines (steep walls, gentle floor), gradient is dominated by steep wall component → GD tends to oscillate across walls instead of moving along the valley floor.

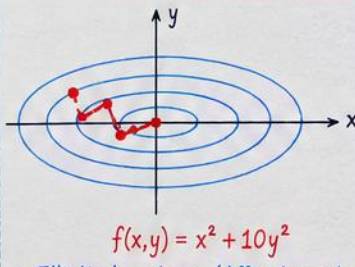
5. Loss Surface Trajectories

(a) Isotropic (well-conditioned) bowl



- Circular contours (same curvature in all directions).
- Gradients point directly toward minimum.
- GD follows smooth path.

(b) Ill-conditioned (anisotropic) surface



- Elliptical contours (different curvature along axes).
- Gradients point away from minimum causing trajectory deflection.
- Successive steps form **right-angle zig-zag path** → slow convergence.

6. Numerical Example

Quadratic bowl : $f(x, y) = x^2 + 10y^2$

$$\nabla f(x, y) = [2x, 20y]^T$$

Start at $(x_0, y_0) = (10, 1)$, $\alpha = 0.08$

k	(x_k, y_k)	$\nabla f(x_k, y_k)$	(x_{k+1}, y_{k+1})
0	(10, 1)	[20, 20]	(8.4, -0.6)
1	(8.4, -0.6)	[16.8, -12]	(7.056, 0.36)
2	(7.056, 0.36)	[14.112, 7.2]	(5.9289, -0.216)
...	...	...	...

- y component keeps flipping sign ($+ \rightarrow - \rightarrow + \dots$).
- Trajectory oscillates across y-axis while making slow progress on x.
- Demonstrates **zig-zagging behavior** in ravines.

7. Condition Number & Ravines

- Condition number: $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ (ratio of max to min curvature).
- High κ → ill-conditioned surface → narrow, elongated valleys.
- Causes **severe zig-zagging** in plain GD.
- **Solutions:** smaller learning rate, momentum, adaptive methods (e.g., Adam), etc.

Key Takeaways

- 1 Gradient descent uses only local slope information.
- 2 ∇f points in direction of steepest ascent; $-\nabla f$ for descent.
- 3 Learning rate controls step size; both direction & magnitude matter.
- 4 Ill-conditioned loss surfaces cause zig-zagging and slow convergence.
- 5 High condition number (κ) indicates anisotropic curvature.

Optimizers with Memory: Momentum, NAG, AdaGrad & Adam

Lecture 07

Use past gradients to get better updates (faster, smoother, adaptive)

1. Why Memory is Needed? (EWMA)

1.1 Limitation of plain SGD

- Uses only current mini-batch gradient g_t
- Ignores past information
- Noisy gradient $\rightarrow$ overreacts (may move in wrong direction)
- Slow in narrow/ill-conditioned valleys

1.2 Exponentially Weighted Moving Average (EWMA)

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

- $\beta \in (0,1)$ controls memory length
- Effective window size $\approx \frac{1}{1-\beta}$ steps
- e.g. $\beta = 0.9 \rightarrow \sim 10$ steps
- More weight to recent gradients (exponential decay)

$\beta \uparrow \Rightarrow$ longer memory
 $\beta \downarrow \Rightarrow$ shorter memory

2. SGD with Momentum & Nesterov (NAG)

2.1 SGD (for reference)

$$w_{t+1} = w_t - \alpha g_t$$

- α : learning rate
- g_t : gradient at step t

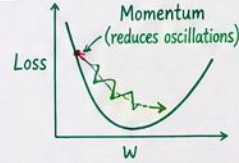
Simple but noisy and slow in ravines.

2.2 SGD with Momentum (Heavy-Ball)

$$v_t = \beta v_{t-1} + g_t$$

$$w_{t+1} = w_t - \alpha v_t$$

- v_t : velocity (accumulates past gradients)
- Accelerates in consistent direction
- Dampens oscillations across steep walls



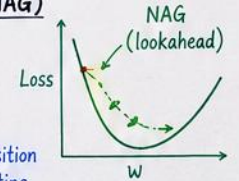
2.3 Nesterov Accelerated Gradient (NAG)

$$g_t = \nabla L(w_t - \alpha \beta v_{t-1})$$

$$v_t = \beta v_{t-1} + g_t$$

$$w_{t+1} = w_t - \alpha v_t$$

- Lookahead: compute gradient at future position
- Detects slope change early $\rightarrow$ less overshooting



3. Adaptive Per-Coordinate Scaling

3.1 AdaGrad

- Scales learning rate by sum of past squared gradients

$$G_t = \sum_{\tau=1}^t g_{\tau}^2$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} g_t$$

- Frequently updated params $\rightarrow$ smaller steps
- Rare params $\rightarrow$ larger steps
- **Drawback**: G_t keeps increasing $\Rightarrow$ learning rate $\rightarrow 0$ (stalls prematurely)

Good for sparse data (e.g. NLP)

3.2 RMSProp

- Uses exponential moving average (fading memory)

$$s_t = \beta_2 s_{t-1} + (1-\beta_2) g_t^2$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{s_t + \epsilon}} g_t$$

- Fixes AdaGrad's stalling
- Learning rate can increase or decrease over time
- Works well for non-stationary problems (RNNs)

Typical $\beta_2 = 0.99$

4. Adam & AdamW

4.1 Adam (Adaptive Moment Estimation)

- Combines Momentum (1st moment) + RMSProp (2nd moment)
- Uses bias correction (since $m_0=0, v_0=0$)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad (1^{st} \text{ moment})$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad (2^{nd} \text{ moment})$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w_{t+1} = w_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Typical values
 $\beta_1 = 0.9$
 $\beta_2 = 0.999$
 $\epsilon = 10^{-8}$

- Fast convergence, low sensitivity to learning rate
- Works well in practice for most deep learning models

4.2 AdamW (Decoupled Weight Decay)

- In standard Adam, L2 regularization is coupled with adaptive gradient (not ideal)
- AdamW applies weight decay directly to parameters

$$w_{t+1} = w_t (1 - \alpha \lambda) - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

- λ : weight decay coefficient (e.g. 0.01)
- Better generalization (especially in Transformers, LLMs)

Use AdamW as default today!

5. Optimizer Selection & Decision Map

5.1 Comparison Table

Optimizer	Update Rule / Idea	Pros	Cons	Typical Use
SGD	$w_{t+1} = w_t - \alpha g_t$	Simple	Noisy, slow	Baseline
SGD + Momentum	$v_t = \beta v_{t-1} + g_t$	Faster, smoother better generalization	Need to tune α, β	Computer Vision (ResNets)
NAG	Lookahead $g_t = \nabla L(w_t - \alpha \beta v_{t-1})$	Less overshoot	Slightly $\uparrow$ compute	CV, large models
AdaGrad	$w_{t+1} = w_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} g_t$	Good for sparse data	Stalls (lr $\rightarrow 0$)	NLP (sparse)
RMSProp	$w_{t+1} = w_t - \frac{\alpha}{\sqrt{s_t + \epsilon}} g_t$	Handles non-stationary data	Needs β_2	RNNs/LSTMs
Adam	$\hat{m}_t, \hat{v}_t$ (bias corr.)	Fast, robust	May generalize worse	Most DL models
AdamW	Decoupled weight decay	Best generalization	Extra hyper-param (λ)	Transformers, LLMs

5.2 General Recommendations

- Use **AdamW** ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \text{weight decay} = 0.01$) as default.
- Use **SGD + Momentum** for vision models (e.g. ResNets).
- Use **RMSProp** for RNNs / non-stationary problems.

6. Worked Numerical Comparison

Gradients: $g_1 = +4, g_2 = +4, g_3 = -1$ ($\alpha = 0.1$)

6.1 SGD

Step	g_t	$\Delta w = -\alpha g_t$	w change
1	+4	-0.4	-0.4
2	+4	-0.4	-0.4
3	-1	+0.1	+0.1

SGD immediately reverses direction on 3rd batch!

6.2 SGD + Momentum ($\beta = 0.9$)

Step	g_t	v_t	$\Delta w = -\alpha v_t$
1	+4	4.0	-0.4
2	+4	$0.9(4.0) + 4 = 7.6$	-0.76
3	-1	$0.9(7.6) - 1 = 5.84$	-0.584

Velocity remains positive (+5.84) $\Rightarrow$ continues in original direction (smooths noise!)

Key Takeaway:

- Momentum (and NAG) use history to avoid overreacting to noisy gradients.
- Adaptive methods (RMSProp, Adam) scale learning rates per parameter.
- **AdamW is the recommended default for modern deep learning.**

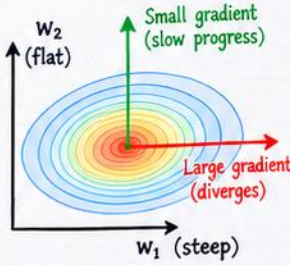
Lecture 08 : Adaptive Learning Rates

AdaGrad, RMSProp & Adam

Deep Learning
Foundations

①. Why One Learning Rate Fails ?

- Loss surface is anisotropic (different curvature in different directions).
- Steep direction (w_1): high curvature, large gradients ($|g_1| \gg 0$).
- Flat direction (w_2): low curvature, tiny gradients ($|g_2| \approx 0$).
- Single global learning rate α cannot work for both:
 - Large $\alpha \rightarrow$ diverges on steep dir.
 - Small $\alpha \rightarrow$ very slow on flat dir.



②. Core Principle of Adaptive Step Sizing

- Measure historical gradient magnitude for each parameter independently.
- Scale step size inversely proportional to its (root mean square) gradient.
- If gradients are large $\rightarrow$ smaller step.
- If gradients are small $\rightarrow$ larger step.

Stop using raw gradient magnitude to determine step distance.

③. AdaGrad : History-Based Scaling

- Accumulate sum of squared gradients for each parameter.
- Larger gradients $\rightarrow$ larger denominator $\rightarrow$ smaller step size.
- Works well for sparse features (e.g. text, recommendation).

Formula

$$G_t = G_{t-1} + g_t^2 = \sum_{\tau=1}^t g_\tau^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot g_t$$

High vs Rare Features

- Frequent features: G_t grows fast $\rightarrow$ small learning rate.
- Rare features: G_t remains small $\rightarrow$ large exploratory steps.

④. Fatal Flaw of AdaGrad

- G_t is monotonically increasing ($g_t^2 \geq 0$).
- Effective learning rate $\alpha/\sqrt{G_t} \rightarrow 0$ as $t \rightarrow \infty$.
- Even when large steps are needed later, AdaGrad remains frozen.
- Cannot effectively navigate non-convex saddles in deep learning.

$$G_1 \leq G_2 \leq G_3 \leq \dots \leq G_t \rightarrow \infty$$

$$\lim_{t \rightarrow \infty} \frac{\alpha}{\sqrt{G_t + \epsilon}} = 0$$

AdaGrad
step size
vanishes

⑤. RMSProp : Leaky Averaging of Squared Gradients

- Uses exponentially weighted moving average (EWMA).
- Older gradients fade out (fading memory).
- Can both increase and decrease step size dynamically.
- Ensures progress along steady gradients regardless of magnitude.

Formula

$$S_t = \beta S_{t-1} + (1 - \beta) g_t^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \odot g_t$$

β : decay parameter
(0.9 or 0.99)
Memory window
 $\approx 1/(1 - \beta)$
(~ 10 steps for 0.9)

Scale Invariance (steady gradient g)

$$S_t \rightarrow g^2 \Rightarrow \Delta w = -\alpha \frac{g}{|g|} = -\alpha \cdot \text{sign}(g)$$

Step size is exactly α , regardless of gradient size !

⑥. Bias Correction & Adam

- Moment vectors initialized at zero ($m_0 = 0, v_0 = 0$).
- Early estimates are biased low.
- Use bias correction factors $(1 - \beta^t)$ to obtain unbiased estimates.
- Combines momentum (1st moment) + RMSProp (2nd moment) + bias correction.

Adam Moment Equations

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Why needed?

$$v_1 = (1 - \beta)(1)^2 = 0.001 \text{ (for } \beta = 0.999)$$

Underestimates true value (1.0) by 1000x

Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Divide by $(1 - \beta^t)$ to correct bias.

⑦. Complete Adam & AdamW Update Rules

Adam Algorithm

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$
- $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$
- $w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$

Standard Hyperparameters

- Learning rate $\alpha = 0.001$ (1e-3)
- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\epsilon = 10^{-8}$

AdamW (Decoupled Weight Decay)

$$w_{t+1} = w_t (1 - \alpha \lambda) - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Decouples weight decay from adaptive scaling. Standard L2 regularization is broken in Adam.

⑧. Optimizer Selection Matrix

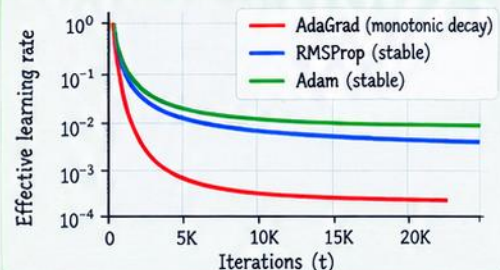
Optimizer	Primary Mechanism	Key Strength	Main Limitation	Application
SGD	Instantaneous gradient	Simple, minimal memory	Oscillates in ravines, sensitive to α	Baseline theory
SGD + Momentum	Velocity accumulation	Better generalization	Needs α, β tuning	CNNs, ResNets
AdaGrad	Sum of squared grads	Auto-tunes sparse features	LR vanishes (monotonic)	Shallow NLP, sparse models
RMSProp	EWMA of squared grads	Overcomes stalling	Noisy numerator gradient	RNNs, Reinforcement Learning
AdamW	Dual moments + bias corr. + decoupled decay	Fast, robust, optimal generalisation	Requires 3x memory	LLMs, Transformers, modern DL

⑨. Numerical Walkthrough (Two Parameters)

Initial: $w_1 = 1.0000, w_2 = 1.0000$, Learning rate: $\alpha = 0.1$
Gradients: w_1 (steep): $g_1 = [3.0, 4.0]$ w_2 (flat): $g_2 = [0.3, 0.4]$

Algorithm	w_1 after Step 1	w_2 after Step 1	w_1 after Step 2	w_2 after Step 2	Ratio $\Delta w_1 / \Delta w_2$
Plain SGD	0.7000	0.9700	0.3000	0.9300	10.0
AdaGrad	0.9000	0.9000	0.8200	0.8200	1.0
RMSProp	0.6838	0.6838	0.4261	0.4261	1.0

⑩. Comparative Trajectories (Step Size vs Iterations)



DEEP LEARNING FOUNDATIONS - LECTURE 09

REGULARISATION : LEARNING THE PATTERN, NOT THE NOISE

1 THE NATURE OF OVERFITTING & GENERALISATION

Generalisation vs Memorisation

Student A (Memoriser)

- ✓ Memorises exact answers to past papers
- ✗ 100% on past papers
- ✗ Fails on new questions

Student B (Learner)

- ✓ Learns underlying concepts
- ✓ 98% on past papers
- ✓ Excels on real exam

Root Cause: Runaway Large Weights

- Pre-activation: $z = \sum w_i x_i + b$
- Large $|w_i| \rightarrow$ tiny input changes create massive output changes
- Jagged, high-frequency decision boundaries
- Small weights $\rightarrow$ smooth, well-behaved boundaries

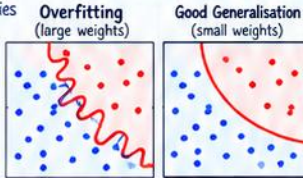
Signal vs Noise

Data = Signal + Noise

- Signal: True underlying relationship (repeats)
- Noise: Measurement error, missing variables, random flukes (does not repeat)

Three ingredients of Overfitting

- 1 Excessive model capacity (too many parameters)
- 2 Noisy or small dataset (limited samples, high error)
- 3 Prolonged training (optimizer fits noise/outliers)



2 L2 REGULARISATION & WEIGHT DECAY (RIDGE)

L2 Penalised Objective

$$J(w) = L(w) + \frac{\lambda}{2} \|w\|_2^2 = L(w) + \frac{\lambda}{2} \sum_{i=1}^D w_i^2$$

- $\lambda > 0$: regularisation strength
- $\lambda = 0 \rightarrow$ standard training
- $\lambda \rightarrow \infty \rightarrow$ forces all weights to zero

Gradient and Weight Decay Update

$$\nabla_w J(w) = \nabla_w L(w) + \lambda w$$

$$w_{t+1} = w_t - \alpha (\nabla_w L(w_t) + \lambda w_t) = w_t (1 - \alpha \lambda) - \alpha \nabla_w L(w_t)$$

Weight decay: multiplicative factor $(1 - \alpha \lambda) < 1$ shrinks weights towards zero at every iteration.

Why L2 Produces Smooth Boundaries

- Prevents weights from growing large
- Keeps activation slopes bounded (small Lipschitz constant)
- Results in smooth, well-generalising decision boundaries



Never Penalise Biases

- Do NOT include bias parameters b in L2 penalty.
- Biases control the position/threshold, not complexity.

3 L1 REGULARISATION & EXACT SPARSITY (LASSO)

L1 Penalised Objective

$$J(w) = L(w) + \lambda \|w\|_1 = L(w) + \lambda \sum_{i=1}^D |w_i|$$

Sub-gradient Update

$$w_{t+1} = w_t - \alpha \lambda \cdot \text{sign}(w_t) - \alpha \nabla_w L(w_t)$$

L2 vs L1 : Push Towards Zero

L2 Push (λw)

- Proportional to w
- Strong when w is large
- Weaker near zero
- Never exactly zero



L1 Push ($\lambda \cdot \text{sign}(w)$)

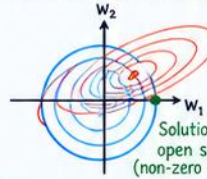
- Constant push
- Same magnitude for all $w \neq 0$
- Drives weights to exact zero



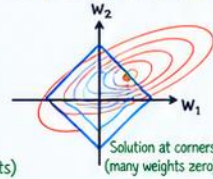
Geometry & Feature Selection

- L1 drives many weights to exactly zero $\rightarrow$ sparse models $\rightarrow$ feature selection
- L2 gives small non-zero weights

L2 (Circular constraint)

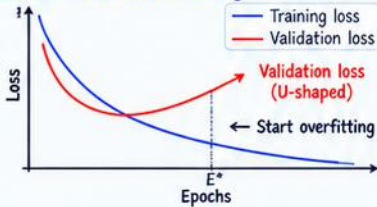


L1 (Diamond constraint)



4 EARLY STOPPING : IMPLICIT REGULARISATION

Validation Loss Monitoring



Stopping Rule (with Patience)

Stop if: $L_{val}(t) > \min_{k \leq t} L_{val}(k)$ for P consecutive epochs

- P = patience (e.g. 5 or 10)
- Accounts for noise in val curve

Restore Best Checkpoint

- Never keep final epoch weights
- Always restore weights from epoch E^* (where validation loss was lowest)

Why It Regularises

- Weights start near zero and only travel finite distance under gradient descent

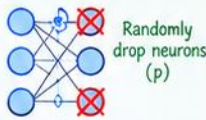
$$\|w_T - w_0\| \leq T \cdot \alpha \cdot G_{max}$$

- Stopping early limits reachable space
- Mathematically equivalent to L2 with $\lambda \propto \frac{1}{T}$ (Sjöberg and Ljung, 1995)
- Zero inference cost; saves training time

5 DROPOUT : ENSEMBLES OF SUBNETWORKS

Neuronal Co-adaptation

- Without dropout, neurons rely on specific partners (co-adaptation)
- Dropout randomly turns off each neuron with probability p (typically 0.5 for hidden, 0.2 for input)
- Trains 2^N subnetworks sharing weights



Inverted Dropout (Training Forward Pass)

$$\tilde{a}_i = \begin{cases} \frac{a_i}{1-p} & \text{with prob. } 1-p \\ 0 & \text{with prob. } p \end{cases}$$

- $E[\tilde{a}_i] = a_i$ (same at train and test)
- No scaling needed at inference

Example ($p = 0.5$)

$a = [2, 4, 6, 8]$
mask $m = [1, 0, 1, 0]$
Scaling = $1/(1-p) = 2.0$
 $\tilde{a} = [4, 0, 12, 0]$
• Original sum = 20
Expected sum = 20

Training vs Inference



Always switch modes in PyTorch

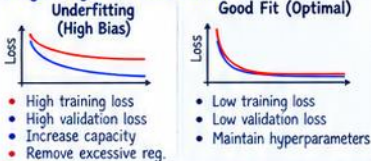
- `model.train()` $\rightarrow$ enables dropout (training loss appears higher)
- `model.eval()` with `torch.no_grad()` $\rightarrow$ disables dropout (validation loss drops)

Key Points

- Prevents co-adaptation of neurons
- Implicit ensemble of 2^N subnetworks
- No inference overhead (disabled at test time)
- Mostly used in fully connected layers, large MLPs, Transformers

6 STRATEGIC REGULARISATION FRAMEWORK

Diagnosing with Loss Curves



Overfitting (High Variance)

- Training loss falling
- Validation loss increasing
- Add early stopping, L2, dropout
- Collect more data

Recommended Recipe

- 1 Steed with high-capacity model that can overfit
- 2 Always use early stopping
- 3 Add moderate L2 weight decay ($\lambda = 10^{-4}$ to 10^{-2})
- 4 If still overfitting, add dropout ($p = 0.2$ to 0.5)

Stacking Regularisers

- Avoid excessive regularisation (diminishing returns)
- Too much regularisation can lead to underfitting

7 STEP-BY-STEP NUMERICAL EXAMPLES

7.1 L2 vs L1 Update Calculation

Given: $w = 0.5$, $\alpha = 0.1$, $\lambda = 0.2$, $\nabla L = 0.05$

L2 Weight Decay

$$w_{t+1} = w_t (1 - \alpha \lambda) - \alpha \nabla L = 0.5 \times 0.98 - 0.005 = \mathbf{0.4850}$$

L1 Update

$$w_{t+1} = w_t - \alpha \lambda \cdot \text{sign}(w_t) - \alpha \nabla L = 0.5 - 0.02 - 0.005 = \mathbf{0.4750}$$

7.2 Inverted Dropout Arithmetic ($p = 0.5$)

$a = [2, 4, 6, 8]$, mask $m = [1, 0, 1, 0]$, scale = 2.0

$\tilde{a} = 2.0 \times [2, 0, 6, 0] = [4, 0, 12, 0]$

- Original sum = 20, Expected sum = 20

8 COMPREHENSIVE COMPARISON MATRIX

Method	Mechanism	Effect on Weights	Inference	Use Case
L2	Penalises $\sum w^2$	Smooth shrinkage	None	Default
L1	Penalises $\sum w $	Exact sparsity	None	Feature selection
Early Stopping	Halt on val min	Limits distance from init	None (saves time)	Universal safety net
Dropout	Random neuron dropout	Prevents co-adaptation	None	FC layers, MLPs, Transformers

Deep Learning Foundations

Lecture 10 Part 1 - CNN Foundations : From Pixels to Patterns

1. Images as Numeric Tensors

- To a computer, an image is a grid of numbers representing light intensities.

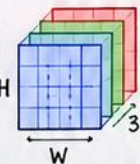
Grayscale Image

- 2D matrix $[H, W]$ (or $[H, W, 1]$)
- Pixel value: 0 (black) - 255 (white)

0	120	255
30	80	200
0	50	120

RGB Image (Color)

- 3D tensor $[H, W, 3]$ (or $[3, H, W]$)
- 3 channels: Red, Green, Blue
- Same spatial coordinates for each channel
- Pixel at (r, c) : $[R_{r,c}, G_{r,c}, B_{r,c}]$

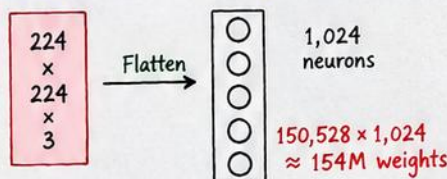


Batch Tensor (PyTorch)

- $X \in \mathbb{R}^{B \times C \times H \times W}$ B: batch size C: channels H: height W: width
- Pixel values normalized: $[0, 255] \rightarrow [0.0, 1.0]$ or $\frac{x - \mu}{\sigma}$ (standardization)

2. Why Fully Connected Layers Fail on Images

- Example**: 224×224 RGB image $\rightarrow$ first hidden layer with 1,024 neurons.
- Input dimension: $224 \times 224 \times 3 = 150,528$
- Weights: $150,528 \times 1,024 \approx 154$ Million params!
- Problems**:
 - Huge parameter count $\rightarrow$ overfitting
 - High memory & computation
 - Loses 2D spatial structure (image becomes 1D)



3. The 2D Convolution Operation

- A small learnable **kernel (filter)** slides over the image.
- At each position, element-wise product + sum + bias.

$$y_{i,j} = \sum_{m=-k}^k \sum_{n=-k}^k x_{i+m,j+n} \cdot w_{m,n} + b$$

- In deep learning, this is **cross-correlation** (kernel not flipped).
- Kernel weights are learned, so flip is not needed.

Sliding Kernel (3×3)

Input patch		
x_{11}	x_{12}	x_{13}
x_{21}	x_{22}	x_{23}
x_{31}	x_{32}	x_{33}

Kernel		
w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

output $y_{i,j}$

- Weight Sharing**: same kernel used at all locations ($3 \times 3 \rightarrow$ only 9 weights + 1 bias).
- Translation Equivariance**: if object shifts by $(\Delta x, \Delta y)$, feature map shifts by same amount $\rightarrow f(T(x)) = T(f(x))$.

4. Spatial Geometry : Padding & Stride

- Valid Padding** ($P=0$): spatial size shrinks $\rightarrow O = W - K + 1$
- Same Padding**: choose P so that $O = W$ (when $S=1$)

$$P = \frac{K-1}{2} \quad (\text{requires odd } K)$$

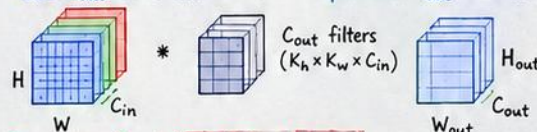
- Examples: $K=3 \rightarrow P=1$ $K=5 \rightarrow P=2$
- Stride (S)**: step size of kernel. $S=2 \rightarrow$ downsample $\sim 1/2$.
- Universal Spatial Output Formula** (for H and W)

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W: input size
K: kernel size
P: padding
S: stride
O: output size

5. Multi-Channel Convolutions & Feature Maps

- Input has C_{in} channels $\rightarrow$ kernel depth must match C_{in} .
- A single filter: shape $[K_h, K_w, C_{in}]$
- Computes one output pixel (sum over all channels) $\rightarrow$ scalar.
- Use C_{out} different filters $\rightarrow$ output with C_{out} channels.



Parameter Count (Convolutional Layer)

$$\text{Params} = (K_h \times K_w \times C_{in} + 1) \times C_{out}$$

(independent of input H, W)

6. Pooling Layers & Downsampling

- Reduce spatial size while preserving important features.
- Max Pooling**: take maximum value in window (captures strongest signal).
- Average Pooling**: take mean of values (smoother).
- Typically 2×2 window with stride 2.

Max Pooling ($2 \times 2, S=2$)

1	3	2	4
5	6	1	2
7	8	0	1
3	2	4	0

 $\rightarrow$

6	4
8	4

Average Pooling ($2 \times 2, S=2$)

1	3	2	4
5	6	1	2
7	8	0	1
3	2	4	0

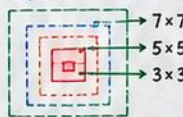
 $\rightarrow$

3.75	2.25
5	1.25

- Provides local translation invariance and noise reduction.
- Downsampling by $2 \times \rightarrow 75\%$ smaller ($4 \times$ less computation).
- Pooling layers have **zero learnable parameters**.
- Operates on each channel independently (channel depth unchanged).

7. Receptive Fields & Hierarchical Features

- Receptive Field (RF)**: spatial area in input that influences a neuron's activation.
- Layer 1 (3×3): RF = 3×3
- Layer 2 (3×3): RF = 5×5
- Layer 3 (3×3): RF = 7×7



Stacking 3×3 vs Single 5×5 (VGG Principle)

- Two 3×3 layers $\rightarrow$ RF = 5×5 , params = $18C_2$
- One 5×5 layer $\rightarrow$ RF = 5×5 , params = $25C_2$
- Two 3×3 uses **28% fewer params** and has two non-linear activations $\rightarrow$ more powerful.

Hierarchical Visual Abstraction

- Early layers**: edges, color blobs, simple corners
- Middle layers**: textures, shapes, circles, stripes
- Deep layers**: object parts (eyes, wheels, handles)
- Final FC layer**: high-level category (car, cat, airplane)

8. Worked Numerical Examples

(a) Manual 3×3 Convolution

Input patch X: $\begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}$ Kernel W: $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$

$b = -1.0$

Dot Product Calculation

Row 1: $1 \times 1 + 0 \times 0 + 0 \times (-1) = 1$
 Row 2: $1 \times 1 + 0 \times 0 + 0 \times (-1) = 1$
 Row 3: $1 \times 1 + 0 \times 0 + 0 \times (-1) = 1$
 Sum = 3
 $y = 3 + (-1.0) = 2.0$
 $\text{ReLU}(2.0) = 2.0$

(b) Padding & Stride Dimension Tracing

Input: 28×28

1) Conv ($K=5, P=2, S=1$)

$$O = \left\lfloor \frac{28 - 5 + 2(2)}{1} \right\rfloor + 1 = 28$$

$\rightarrow 28 \times 28$

2) Max Pool ($K=2, P=0, S=2$)

$$O = \left\lfloor \frac{28 - 2 + 0}{2} \right\rfloor + 1 = 14$$

$\rightarrow 14 \times 14$

(c) Parameter Counting

$C_{in} = 3, C_{out} = 64, K = 3 \times 3$

Weights per filter: $3 \times 3 \times 3 = 27$

Bias per filter: 1

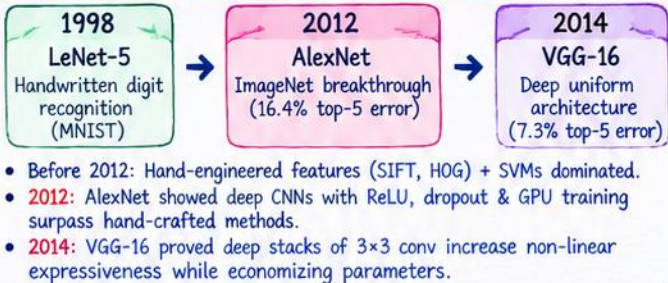
Params per filter: $27 + 1 = 28$

Total params = $28 \times 64 = 1,792$

Deep Learning Foundations

Lecture 10 Part 2 - Classical CNN Architectures: LeNet, AlexNet & VGG-16

1. Historical Evolution (1998 → 2014)



2. Executive Summary

- Transition from **LeNet-5 (1998)** to **AlexNet (2012)** and **VGG-16 (2014)** marks the foundational progression of computer vision.
- AlexNet**: deep networks with **ReLU + Dropout** can be trained on GPUs to surpass hand-crafted algorithms.
- VGG-16**: replacing large filters (11x11, 5x5) with deep cascades of 3x3 convolutions increases non-linear expressiveness while reducing parameters.

3. Architectures at a Glance

Network	Year	Input Size	Conv Filters	Activation	Total Params	Key Milestone
LeNet-5	1998	32 × 32 × 1	5 × 5	Tanh/Sigmoid	≈ 60k	First commercial digit recognizer
AlexNet	2012	224 × 224 × 3	11 × 11, 5 × 5, 3 × 3	ReLU	≈ 60M	ImageNet breakthrough (16.4% top-5 error)
VGG-16	2014	224 × 224 × 3	Uniform 3 × 3 only	ReLU	≈ 138M	Proved depth with small filters beats wide shallow filters

4. Key Formulas

Convolution Output Size

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

W : input size
F : filter size
P : padding
S : stride
O : output size

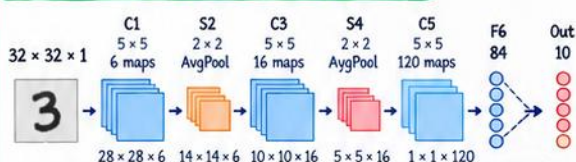
Parameters (Conv Layer)

Params = $(F \times F \times C_{in} + 1) \times C_{out}$
F : filter size C_{in} : input channels
 C_{out} : output channels +1 : bias

Parameters (Fully Connected)

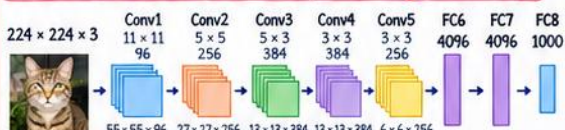
Params = $(N_{in} + 1) \times N_{out}$

5. LeNet-5 (1998) - The Pioneer



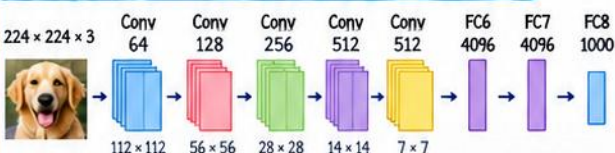
- Input: 32 × 32 grayscale digits (MNIST)
- 5 × 5 convolutions, average pooling, tanh/sigmoid activations
- Total parameters: ≈ **60,000**
- First commercial digit recognizer

6. AlexNet (2012) - Deep Learning Revolution



- Input: 224 × 224 RGB images
- 8 layers (5 Conv + 3 FC)
- ReLU, dropout (p = 0.5), data augmentation, overlapping max pooling
- Total parameters: ≈ **60 million** (95% in FC layers)
- Top-5 error on ImageNet: **16.4%**

7. VGG-16 (2014) - Principle of Factorization



- Input: 224 × 224 RGB images
- 16 layers (13 Conv + 3 FC)
- All convolutions are 3 × 3, stride 1, padding 1
- 2 × 2 max pooling between blocks
- Total parameters: ≈ **138 million**
- Top-5 error on ImageNet: **7.3%**

8. Factorizing Large Filters (VGG Principle)

Single 7 × 7 Convolution

Parameters: 49C²
Receptive field: 7 × 7
Non-linearities: 1

Stack of Three 3 × 3 Convolutions

Parameters: 27C² (45% less)
Receptive field: 7 × 7 (same)
Non-linearities: 3 (3 × ReLU)

9. Computational Bottlenecks & Memory Distribution

Early Layers

- < 10% of total parameters
- > 80% of activation memory and FLOPs
- Feature maps are large (224 × 224, 112 × 112)

Final FC Layers

- > 80-90% of total parameters
- Minimal activation memory
- Example: VGG-16 FC6 has 102.7M weights (≈ 392 MB in FP32)

10. LeNet-5 Layer Details (≈ 60,000 Params)

Layer	Input Shape	Filter Specs	Output Shape	Params
Input	32 × 32 × 1	-	32 × 32 × 1	0
C1	32 × 32 × 1	6 × (5 × 5)	28 × 28 × 6	156
S2	28 × 28 × 6	2 × 2	14 × 14 × 6	0
C3	14 × 14 × 6	16 × (5 × 5)	10 × 10 × 16	1,516
S4	10 × 10 × 16	2 × 2	5 × 5 × 16	0
C5	5 × 5 × 16	120 × (5 × 5)	1 × 1 × 120	48,120
F6	120	Dense	84	10,164
Output	84	Dense	10	850
Total Parameters				≈ 60,000

11. AlexNet Layer Details (≈ 60M Params)

Layer	Output Shape	Params
Conv1 + Pool1	27 × 27 × 96	34,944
Conv2 + Pool2	13 × 13 × 256	614,656
Conv3	13 × 13 × 384	885,120
Conv4	13 × 13 × 384	1,327,488
Conv5 + Pool5	6 × 6 × 256	884,992
FC6	4,096	37,752,832
FC7	4,096	16,781,312
FC8	1,000	4,097,000
Total Parameters		≈ 60,000,000

12. VGG-16 Layer Details (≈ 138M Params)

Stage	Layer Type	Output Shape	Params
Stage 1	2 × Conv 3 × 3	112 × 112 × 64	38,720
Stage 2	2 × Conv 3 × 3	56 × 56 × 128	221,440
Stage 3	3 × Conv 3 × 3	28 × 28 × 256	1,475,328
Stage 4	3 × Conv 3 × 3	14 × 14 × 512	5,899,776
Stage 5	3 × Conv 3 × 3	7 × 7 × 512	7,079,424
FC6	Dense	4,096	102,764,544
FC7	Dense	4,096	16,781,312
FC8	Dense	1,000	4,097,000
Total Parameters			≈ 138,000,000

13. Key Calculations

AlexNet Conv1 MACs

- Input: 224 × 224 × 3
- Filters: 96, Kernel: 11 × 11, S = 4, P = 2
- Output: 55 × 55 × 96
- MACs: 55 × 55 × 96 × 363
- ≈ **105 million operations**

VGG-16 FC6 Memory

- Weights: 25,088 × 4,096 = 102,760,448
- Total (with bias): 102,764,544
- Memory (FP32): 102,764,544 × 4 bytes
- ≈ **392.0 MB**

14. Final Comparison

Metric	LeNet-5	AlexNet	VGG-16
Learned Layers	7	8	16
Total Params	≈ 60K	≈ 60M	≈ 138M
Weight Memory (FP32)	< 1 MB	≈ 240 MB	≈ 528 MB
Input Resolution	32 × 32 × 1	224 × 224 × 3	224 × 224 × 3
Non-linearity	Tanh/Sigmoid	ReLU	ReLU
Top-5 Error	N/A (MNIST)	16.4%	7.3%

Deep Learning Foundations

Lecture 11 - Modern CNN Architectures: Inception & ResNet

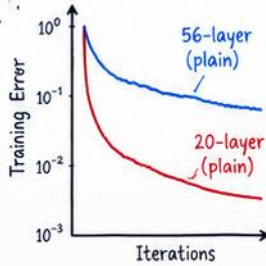
1. Executive Summary & Key Idea

- As networks grow deeper, plain CNNs face the **degradation problem**: deeper networks have higher training and validation error than shallower ones.
- Two architectures solved this at ILSVRC 2014 & 2015:
 - GoogLeNet (Inception)**: multi-scale parallel receptive fields, 1x1 bottlenecks, Global Average Pooling (~6.8M parameters).
 - ResNet**: identity shortcut connections $H(x) = F(x) + x$, enabling very deep networks (50, 101, 152 layers).

2. Limits of Plain ConvNets & Depth Barrier

- Following VGG, researchers tried deeper networks (20, 30, 56 layers).
- Repeated multiplication of Jacobians causes vanishing or exploding gradients.
- Degradation Problem** (He et al., 2015): A 56-layer plain network had higher training error than a 20-layer network (on CIFAR-10). This is an optimization failure, not overfitting.
- Wider vs Deeper**: Increasing width linearly increases parameters and quadratically increases computation. We need architectures that can safely scale deeper.

Degradation Problem



Formula Sheet

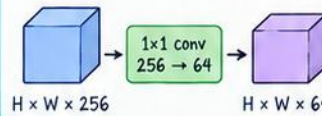
- 1x1 Convolution**:
$$Y_{h,w,k} = \sum_{c=1}^{C_{in}} X_{h,w,c} \cdot W_{k,c} + b_k$$
 Preserves spatial $H \times W$
Changes $C_{in} \rightarrow C_{out}$
- Residual Block**: $H(x) = F(x, \{W_i\}) + x$
- Residual Mapping**: $F(x) = H(x) - x$
- Gradient Highway**: $\frac{\partial E}{\partial x} = \frac{\partial E}{\partial H} \left(\frac{\partial F}{\partial x} + I \right) = \frac{\partial E}{\partial H} \frac{\partial F}{\partial x} + \frac{\partial E}{\partial H}$
- Projection Shortcut**: $H(x) = F(x) + W_s x$ (W_s : 1x1 conv, stride $S=2$)
- ResNet Bottleneck**:
1x1 conv (reduce C) $\rightarrow$ 3x3 conv $\rightarrow$ 1x1 conv (restore 4C)

3. The 1x1 Convolution as a Dimensionality Reducer

- Introduced by Lin et al, (2013) in Network in Network.
- Acts as a local MLP across channel depth at each spatial pixel.

$$Y_{h,w,k} = \text{ReLU} \left(\sum_{c=1}^{C_{in}} X_{h,w,c} \cdot W_{k,c} + b_k \right)$$

- Shrinks channel depth ($C_{in} \rightarrow C_{out}$) while preserving spatial height and width ($H \times W$).
- Used as a bottleneck before expensive 3x3 or 5x5 conv to reduce FLOPs by 70% - 80%.



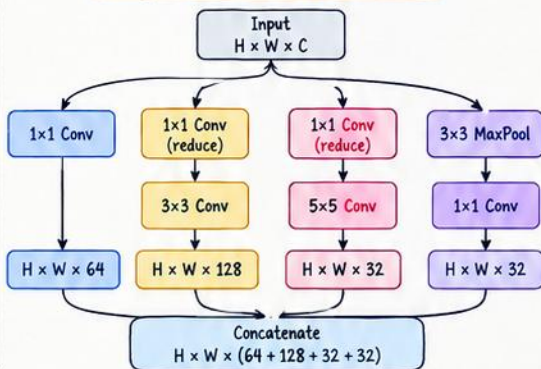
Benefits:

- Reduces FLOPs
- Preserves spatial size
- Cross-channel projection
- Adds non-linearity (ReLU)

4. GoogLeNet / Inception Architecture

- Applies 1x1, 3x3, 5x5 conv and 3x3 MaxPool in parallel.
- Uses 1x1 bottlenecks to keep computation low.
- Uses Global Average Pooling (GAP) instead of dense FC layers (6.8M parameters).
- Uses auxiliary classifiers during training to combat vanishing gradients (discarded at inference).

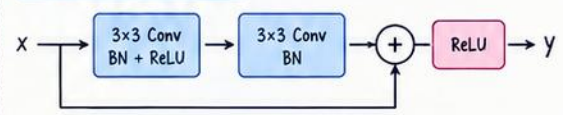
Inception Module (with Bottlenecks)



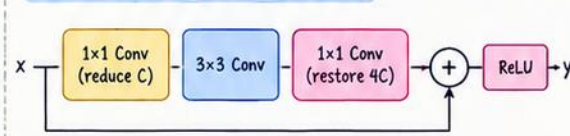
5. Deep Residual Learning (ResNet)

- Instead of learning $H(x)$, learn residual mapping $F(x) = H(x) - x$.
- Residual block: $H(x) = F(x) + x$
- Identity shortcuts allow easy optimization and enable very deep networks (50, 101, 152 layers).
- Provides an unobstructed gradient highway.

Residual Block (Basic)

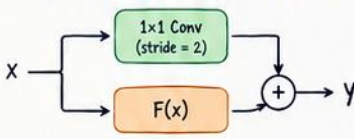


Bottleneck Block (ResNet-50/101/152)



6. Projection Shortcut (when shapes differ)

- When spatial dimensions shrink (stride $S=2$) or channel depth changes, we use a 1x1 projection shortcut.



7. Dimension Tracing: ResNet-50

Stage	Output Spatial Shape	Bottleneck Structure	Repeats	Output Channels
Conv1	112 x 112	$7 \times 7, S=2 \rightarrow$ MaxPool $3 \times 3, S=2$	1	64
Stage 2	56 x 56	$[1 \times 1, 64] \rightarrow [3 \times 3, 64] \rightarrow [1 \times 1, 256]$	3	256
Stage 3	28 x 28	$[1 \times 1, 128] \rightarrow [3 \times 3, 128] \rightarrow [1 \times 1, 512]$	4	512
Stage 4	14 x 14	$[1 \times 1, 256] \rightarrow [3 \times 3, 256] \rightarrow [1 \times 1, 1024]$	6	1024
Stage 5	7 x 7	$[1 \times 1, 512] \rightarrow [3 \times 3, 512] \rightarrow [1 \times 1, 2048]$	3	2048
Classifier	1 x 1	Global Average Pooling $\rightarrow$ Linear (1048 $\rightarrow$ 1000)	1	1000

8. Inception FLOP Reduction (Example)

- Input: $28 \times 28 \times 192$, 5x5 conv $\rightarrow$ 32 output channels

Without 1x1 Bottleneck

MACs = $28 \times 28 \times 32 \times (5 \times 5 \times 192)$
 $\approx 120,422,400$
(120.4 Million)

With 1x1 Bottleneck (192 $\rightarrow$ 16)

1) 1x1: $28 \times 28 \times 16 \times (1 \times 1 \times 192) = 2,408,448$
2) 5x5: $28 \times 28 \times 32 \times (5 \times 5 \times 16) = 10,035,200$
Total = 12,443,648 (12.4 Million)

$\sim 10 \times$
less FLOPs

9. ResNet Bottleneck Block Parameters

Input channels = 256, bottleneck = 64, output = 256

Layer	Calculation	Parameters
1x1 Conv (256 $\rightarrow$ 64)	$(1 \times 1 \times 256 + 1) \times 64$	16,448
3x3 Conv (64 $\rightarrow$ 64)	$(3 \times 3 \times 64 + 1) \times 64$	36,928
1x1 Conv (64 $\rightarrow$ 256)	$(1 \times 1 \times 64 + 1) \times 256$	16,640
Total Parameters		70,016

10. Architecture Comparison

Architecture	Year	Depth	Params	FLOPs	Top-5 Error
AlexNet	2012	8	60 M	0.72 G	16.4%
VGG-16	2014	16	138 M	15.3 G	7.3%
GoogLeNet	2014	22	6.8 M	1.5 G	6.7%
ResNet-50	2015	50	25.6 M	3.8 G	5.25%